

## Seven day plan

---

Aggressive but survivable. Roughly 6 hours a day. If a day runs over, cut the stretch task, never the checkpoint.

Each day follows the same loop: **\*\*concept**, your attempt, my review, production solution, **commit.\*\*** You write the important pieces. I review and then show you what a production version looks like and why it differs from yours.

---

## Day 1 – Messy data and profiling

---

**Detail:** `day01.md`

- Why profiling comes before cleaning
- Loud, medium and subtle faults
- Referential integrity, whitespace on join keys, null versus wrong
- **Build:** `profile_dataframe()` and a profile report over all five files
- **Checkpoint:** profile runs, findings written, branch committed
- **Then:** score your profiler against the fault log

## Day 2 – SQL, the canonical model and source mapping

---

- DuckDB, loading CSV, why it beats pandas for joins at this size
- Canonical schema design, mapping four ERPs onto one model
- SELECT, WHERE, CASE, joins, GROUP BY, HAVING, CTEs
- Window functions: ROW\_NUMBER, RANK, LAG, LEAD
- Duplicate detection, conditional aggregation, NULL semantics
- **Build:** staging and canonical tables, source mapping config
- **Practice:** 20 SQL questions on this data, you answer first
- **Benchmark:** pandas vs Polars vs DuckDB, measured not asserted

## Day 3 – The rules engine

---

- Decision tables, precedence, effective dating, rule versioning
- Configuration versus code, and where that boundary should sit
- **Build:** `tax_rules.yaml` and the engine that evaluates it
- **Build:** the calculation spec, written *before* the code
- Tax-inclusive vs exclusive, rounding, currency, zero-rated vs exempt
- **Simulation:** requirements workshop, I play the Tax SME and give you a

vague requirement. You ask the questions.

## Day 4 – Data quality, reconciliation, anomalies

---

- Twelve named DQ checks with severity, threshold and status
- Golden dataset: what it is, why it is signed off before build
- Reconciliation: MATCH, ROUNDING\_DIFFERENCE, RATE\_MISMATCH, DATA\_ERROR,

RULE\_ERROR, MISSING\_DATA, DUPLICATE, UNKNOWN

- Anomaly detection: z-score, IQR, percentile, plus deterministic rules
- **Build:** DQ framework, reconciliation engine, exception register
- **Measure:** your precision and recall against the fault log

## Day 5 – Testing and the production incident

---

- pytest structure, fixtures, parametrisation, coverage of business logic
- Regression testing: change one rule, prove nothing else moved
- Property-based thinking on tax invariants
- **Incident:** I give you symptoms only. You investigate with SQL and logs,

find root cause, fix, rerun regression, verify, and write the incident report. I do not tell you the answer.

## Day 6 – Git, CI/CD, docs, AI evaluation

---

- Feature branches, meaningful commits, a PR you would approve
- GitHub Actions: lint, test, coverage, fail on critical
- Logging and observability, what to log and what never to log
- Data dictionary and technical decisions record
- **Build:** AI evaluation harness. Natural language rule to structured

proposal to deterministic validation to human approval. PASS, FAIL, REQUIRE\_HUMAN\_REVIEW. Hallucinations, ambiguity, precedence, safety.

## Day 7 – Interview

---

- Full mock interview, graded 1 to 5 per answer
- Python debugging simulations, deliberately broken code
- SQL investigation simulations, symptoms only
- Code review simulations, mediocre code to critique
- Behavioural questions using your real Stripe examples
- **Produce:** resume bullets, 30 second, 60 second and 2 minute explanations,

and the final knowledge matrix

---

## Priorities if you fall behind

---

Do these no matter what: **Day 2 SQL, Day 3 rules engine**, \*\*Day 4 reconciliation, Day 5 incident\*\*. Those four are the job.

Drop first: the Streamlit dashboard, the Polars benchmark, the AI harness beyond a working skeleton.